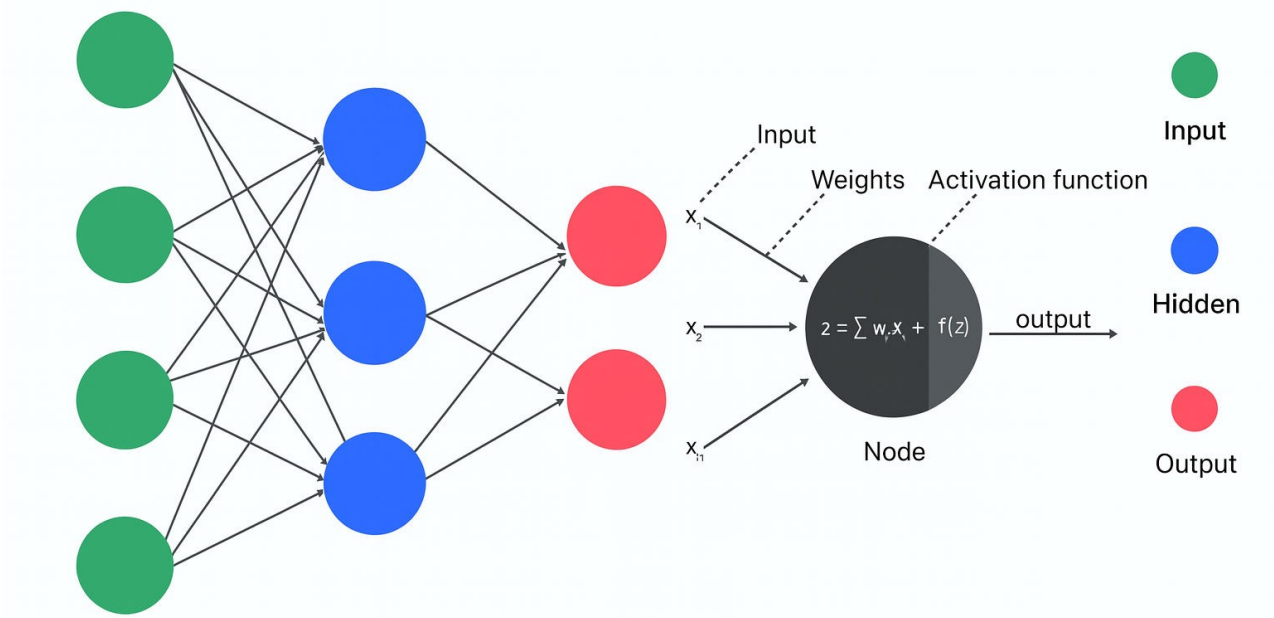
KIET Deemed to be University

Department of Computer Science & Engineering

Presentation on

# Training Neural Networks




**Presented By:**

Dr. Dharmendra Pathak

Associate Professor

CSE Department

# Introduction to Deep Learning

- Deep Learning is a branch of Machine Learning that uses neural networks with multiple hidden layers to learn patterns directly from data. It is especially useful for unstructured data such as images, speech, video, and text.

- Mimics human brain for intelligent tasks.
- Employs layers for automatic feature extraction.
- Revolutionized AI with predictive performance.
- Requires substantial data for accurate results.
- Utilizes neural networks for solving problems.
- Applications include healthcare, finance, image recognition.

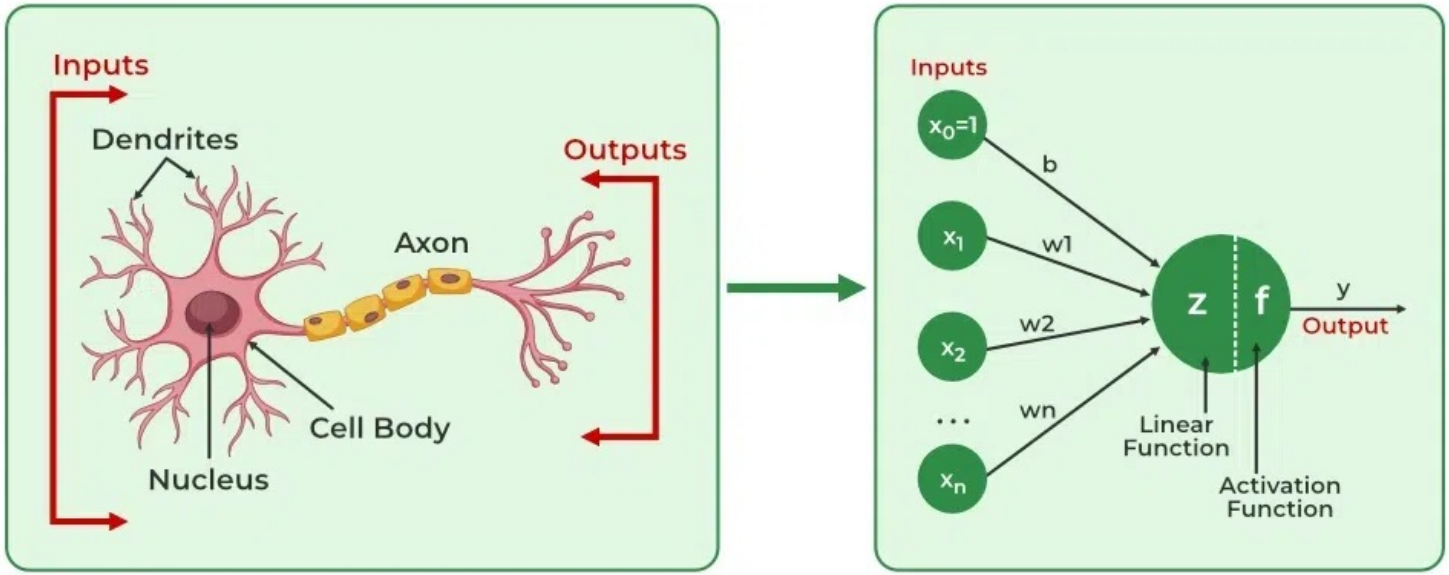
# Scope of Deep Learning

- Computer Vision: image classification, object detection, face recognition
- Natural Language Processing: translation, chatbots, sentiment analysis, text generation
- Speech: speech recognition and voice assistants
- Healthcare: medical-image analysis and disease prediction
- Finance: fraud detection, risk prediction, algorithmic trading
- Generative AI: generating text, images, music, video, and code
- Autonomous systems: self-driving cars, robotics, and drones

# Neuron Models

- Inspired by biological neurons
- Basic Components:
  - Inputs ($x_1$, $x_2$, … xn)
  - Weights ($w_1$, $w_2$, … wn)
  - Bias (b)
  - Activation function
- Mathematical Model:
- $y = f(\sum w_i x_i + b)$

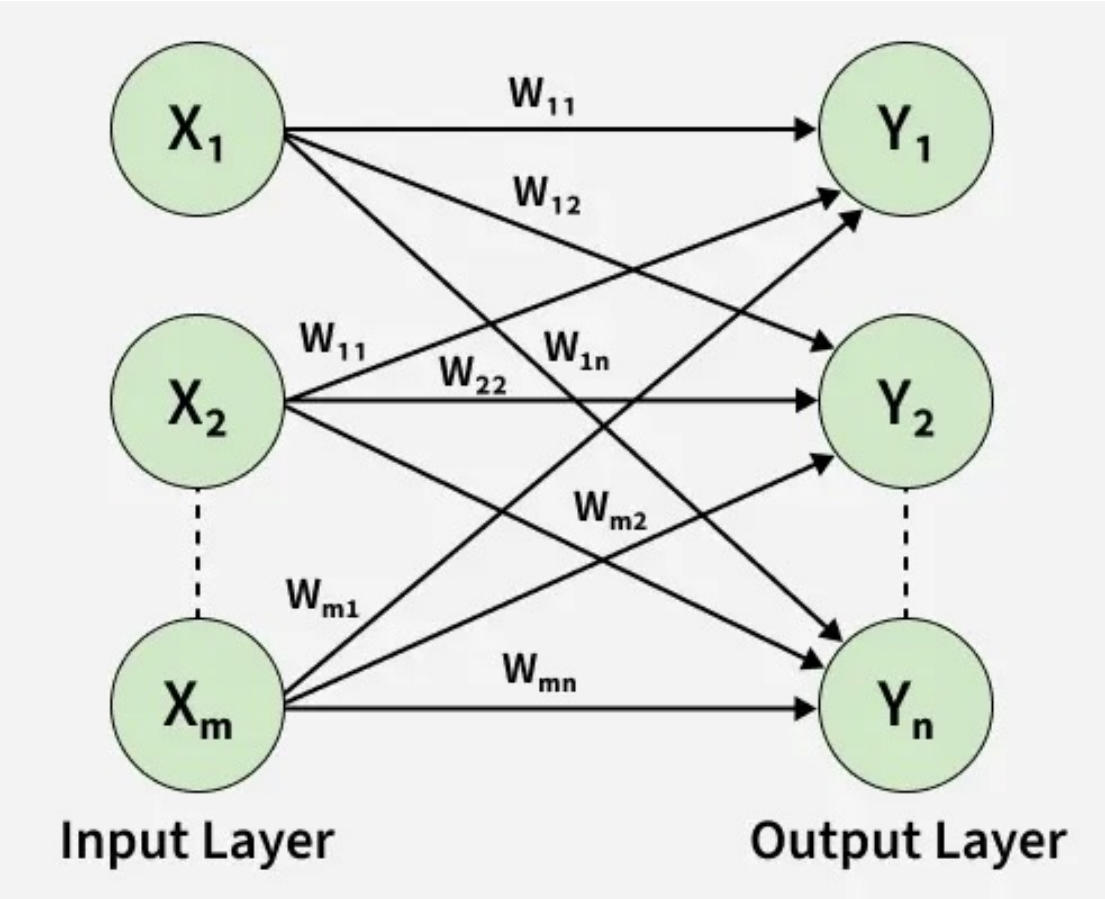
# Neuron Models



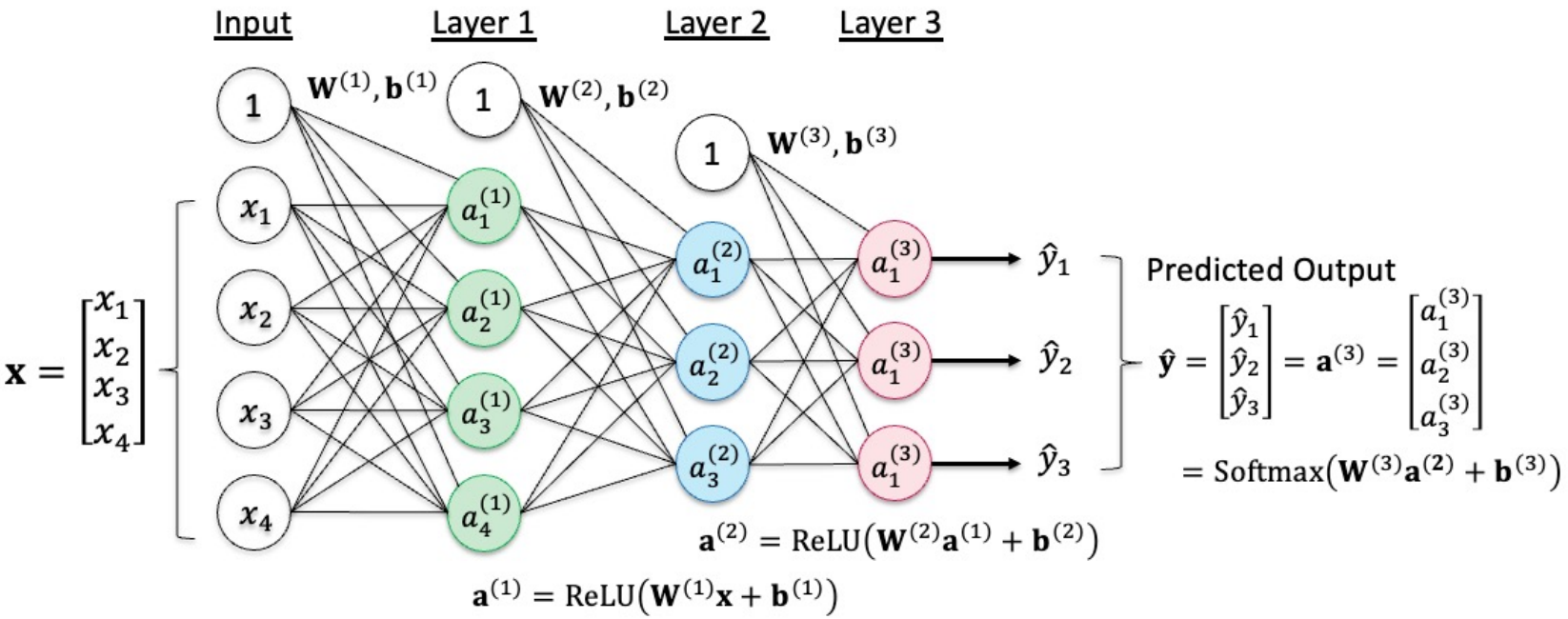
# Network Architectures

- Definition: Arrangement of neurons in a network
- Types:
  - Single Layer Network
  - Multi-Layer Network
  - Feedforward Network
  - Recurrent Network
- Key Concept:
  - Layers: Input → Hidden → Output

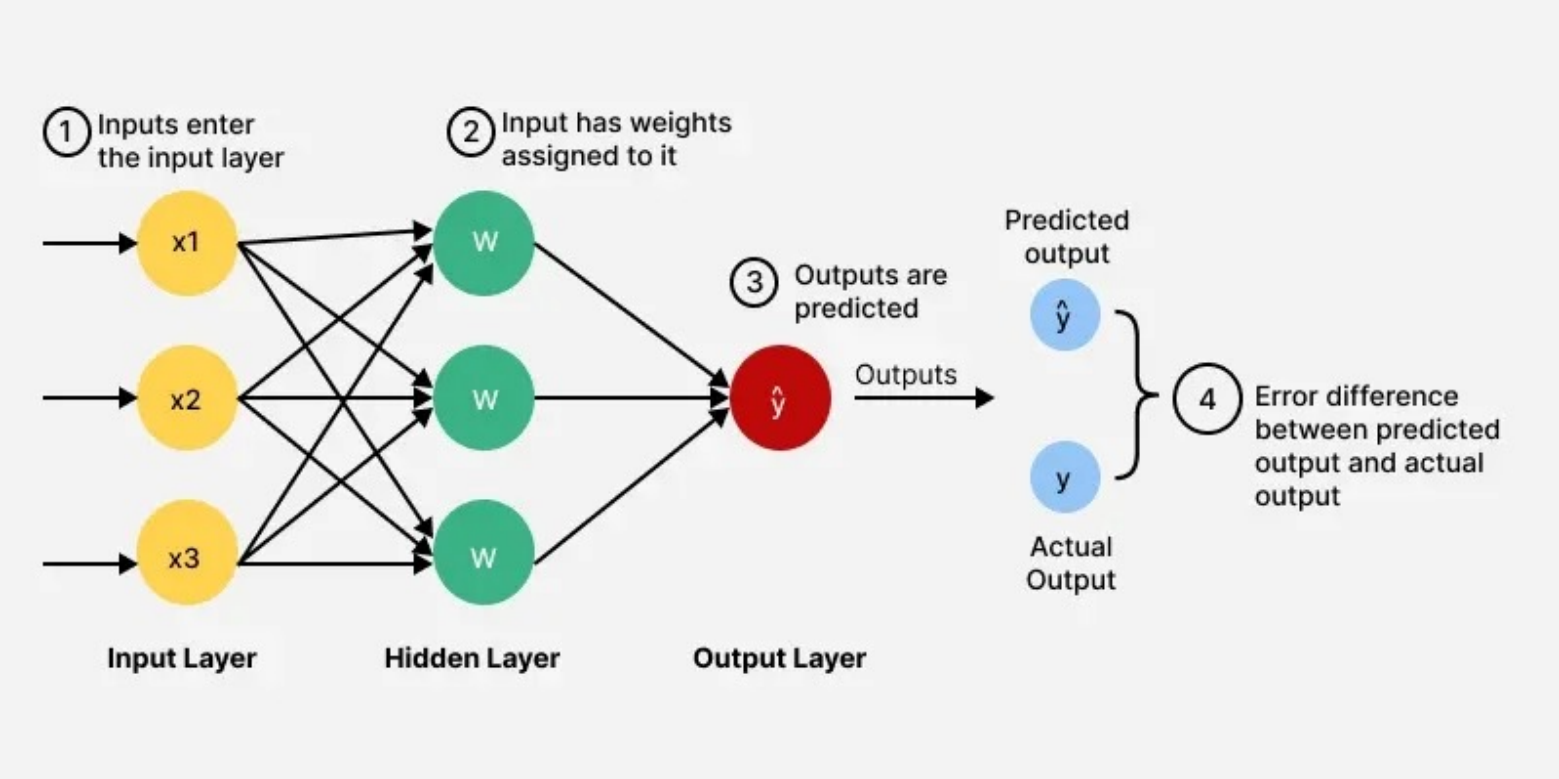
# Network Architectures: Single Layer Network



# Network Architectures: Multi-Layer Network

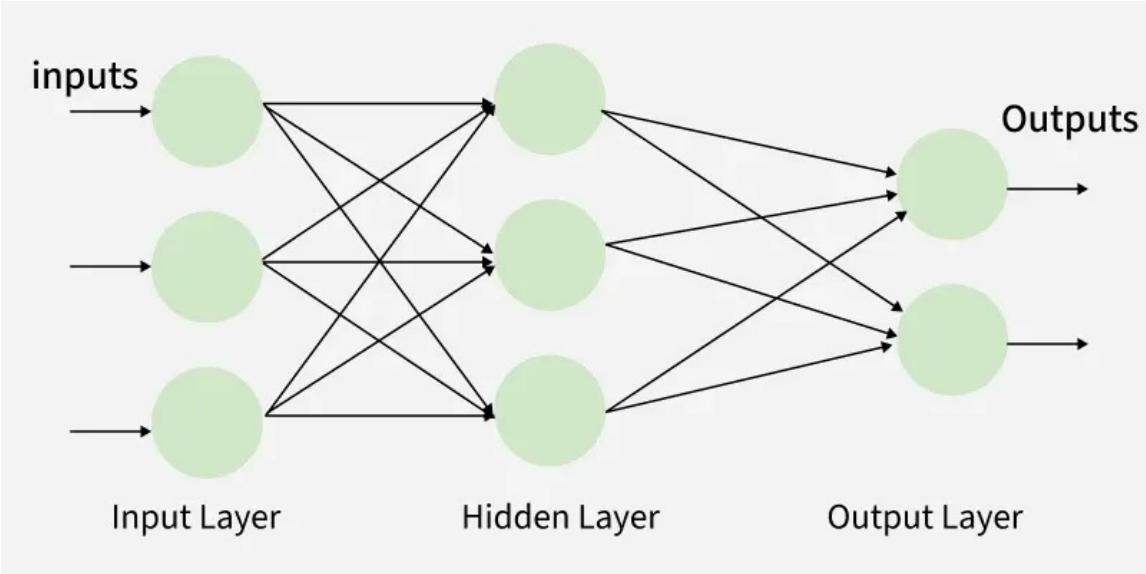


# Network Architectures: Feedforward Network



# Multilayer Perceptron Network

- Multi-Layer Perceptron (MLP) consists of fully connected dense layers that transform input data from one dimension to another.
- It is called multi-layer because it contains an input layer, one or more hidden layers and an output layer.
- The purpose of an MLP is to model complex relationships between inputs and outputs.



# Multilayer Perceptron Network: Working

- Forward Propagation

In forward propagation the data flows from the input layer to the output layer, passing through any hidden layers. Each neuron in the hidden layers processes the input as follows:

Weighted Sum: The neuron computes the weighted sum of the inputs:

$$z = \sum_i w_i x_i + b$$

Where:

$x_i$ is the input feature.

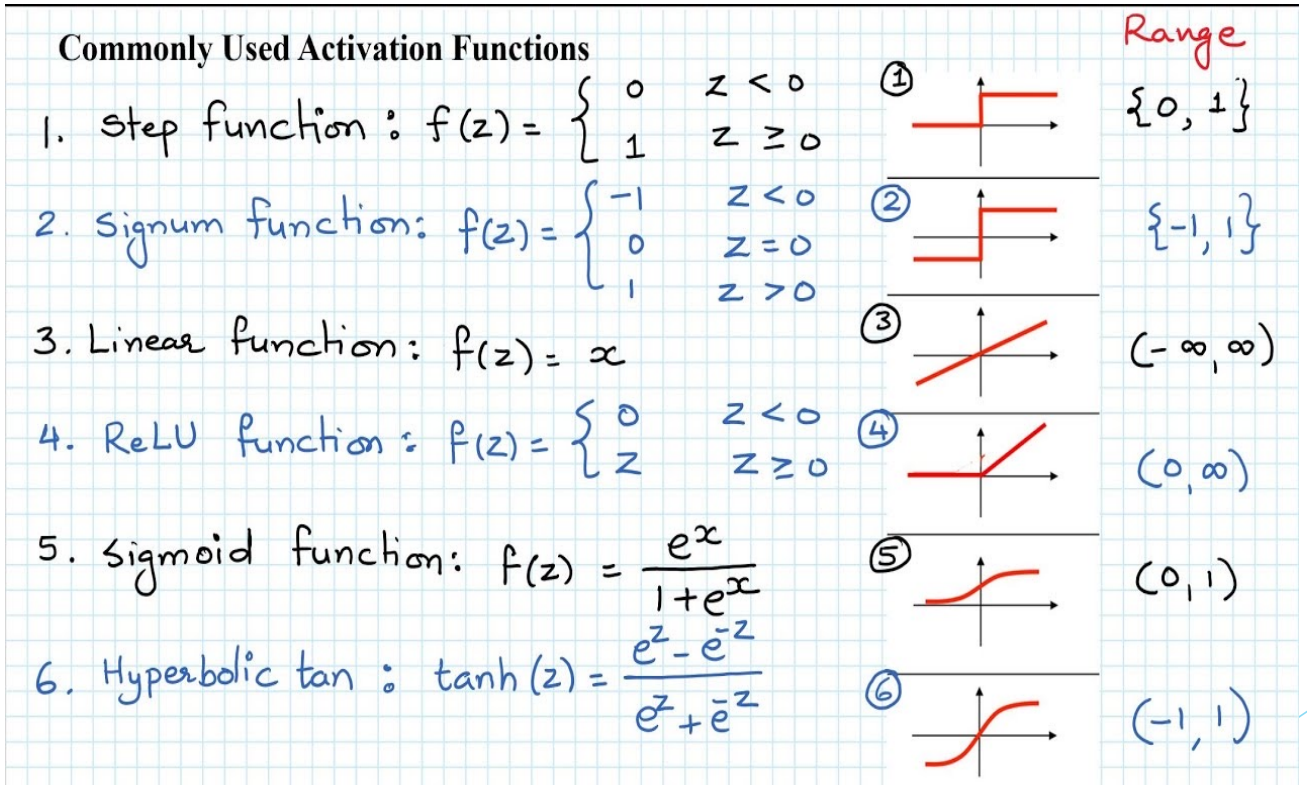
$w_i$ is the corresponding weight.

$b$ is the bias term.

# Multilayer Perceptron Network: Working

- Activation Function

The weighted sum z is passed through an activation function to introduce non-linearity.



# Multilayer Perceptron Network: Working

- Loss Function

- Once the network generates an output the next step is to calculate the loss using a loss function. In supervised learning this compares the predicted output to the actual label.

- For regression problems the mean squared error (MSE) is often used:

$$MSE = \frac{1}{N}\sum_{i=1}^{N}(y_i - \hat{y}_i)^2$$

# Multilayer Perceptron Network: Working

- Back Propagation
- The goal of training an MLP is to minimize the loss function by adjusting the network's weights and biases. This is achieved through backpropagation:
- Gradient Calculation: The gradients of the loss function with respect to each weight and bias are calculated using the chain rule of calculus.
- Error Propagation: The error is propagated back through the network, layer by layer.
- Gradient Descent: The network updates the weights and biases by moving in the opposite direction of the gradient to reduce the loss: $w = w - \eta \cdot \frac{\partial L}{\partial w}$

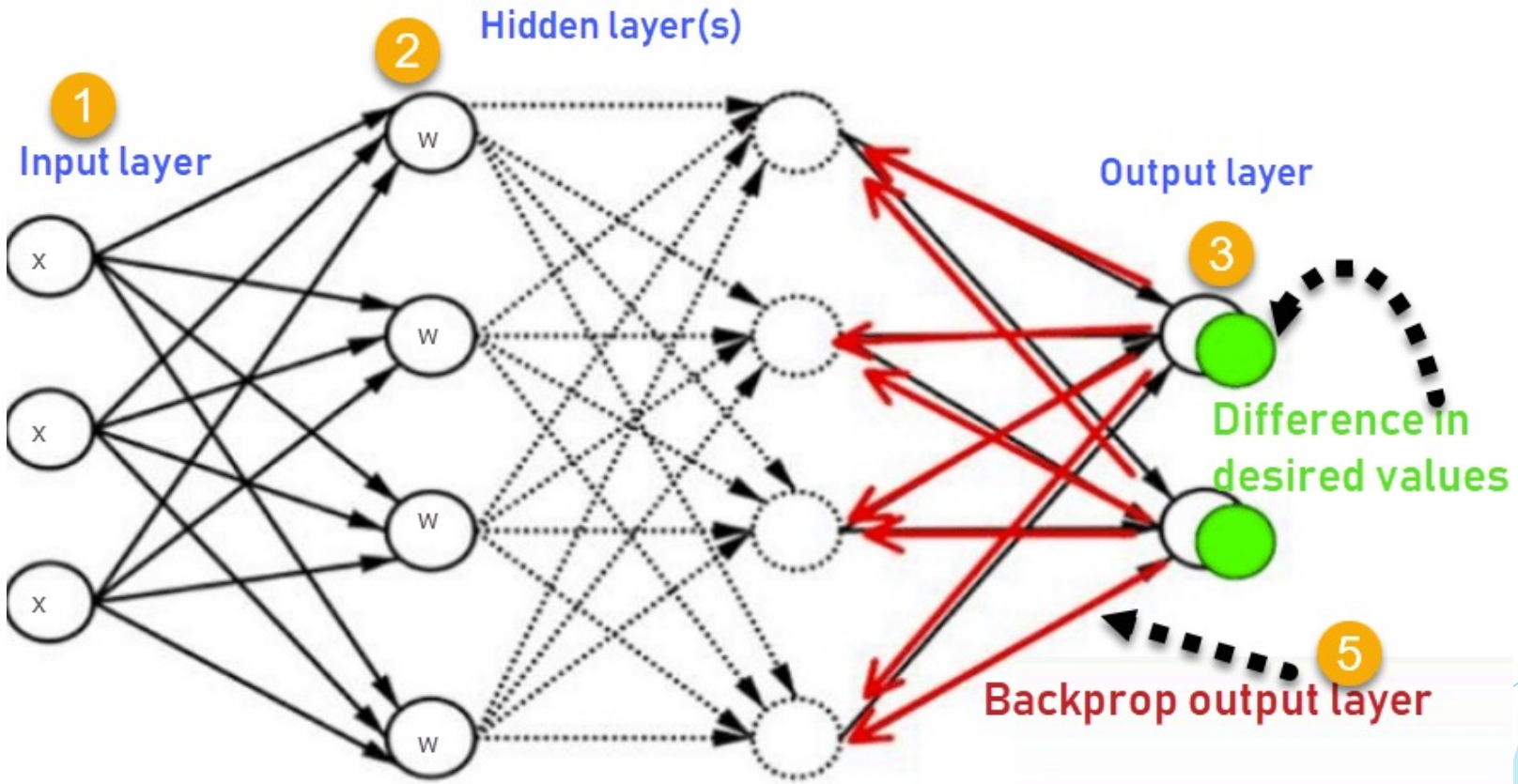
where: $w$ is the weight, $\eta$ is the learning rate

$\frac{\partial L}{\partial w}$ is the gradient of the loss function with respect to the weight.

# Introduction to Back Propagation Network

- Back propagation is a supervised learning technique for neural networks that calculates the gradient of descent for weighting different variables.
- It's short for the backward propagation of errors, since the error is computed at the output and distributed backwards throughout the network's layers.
- When an artificial neural network discovers an error, the algorithm calculates the gradient of the error function, adjusted by the network's various weights.
- The gradient for the final layer of weights is calculated first, with the first layer's gradient of weights calculated last. Partial calculations of the gradient from one layer are reused to determine the gradient for the previous layer.
- This point of this backwards method of error checking is to more efficiently calculate the gradient at each layer than the traditional approach of calculating each layer's gradient separately.

# Introduction to Back Propagation Network



# Back Propagation Network Algorithm

- **Step 1 :**

  Normalize the I/P and O/P with respect to their maximum values. For each training pair, assume that in normalized form there are

  $\ell$ inputs given by $\{ I \}_I$ and

  $\ell$ x 1

  **n** outputs given by $\{ O \}_O$

  n x 1

- **Step 2 :**

  Assume that the number of neurons in the hidden layers lie between $1 < m < 21$

# Back Propagation Network Algorithm

- **Step 3 :**

  Let [ **V** ] represents the weights of synapses connecting input neuron and hidden neuron

  Let [ **W** ] represents the weights of synapses connecting hidden neuron and output neuron

  Initialize the weights to small random values usually from **-1 to +1;**

  $[V]^0 = [\text{random weights}]$

  $[W]^0 = [\text{random weights}]$

  $[\Delta V]^0 = [\Delta W]^0 = [0]$

  For general problems $\lambda$ can be assumed as **1** and threshold value as **0**.

# Back Propagation Network Algorithm

- **Step 4 :**

  For training data, we need to present one set of inputs and outputs.
  Present the pattern as inputs to the input layer $\{ I \}_I$ .
  then by using linear activation function, the output of the input layer may be evaluated as

$$\underset{\ell \times 1}{\{ O \}_I} = \underset{\ell \times 1}{\{ I \}_I}$$

- **Step 5 :**

  Compute the inputs to the hidden layers by multiplying corresponding weights of synapses as

$$\underset{m \times 1}{\{ I \}_H} = \underset{m \times \ell}{[ V ]^T} \underset{\ell \times 1}{\{ O \}_I}$$

- **Step 6 :**

  Let the hidden layer units, evaluate the output using the sigmoidal function as

$$\{ O \}_H = \left\{ \begin{array}{c} - \\ - \\ \dfrac{1}{( 1 + e^{-(IH)} )} \\ - \\ - \end{array} \right\}$$

# Back Propagation Network Algorithm

- **Step 7 :**

  Compute the inputs to the output layers by multiplying corresponding weights of synapses as

$$\{ I \}_O = [ W ]^T \{ O \}_H$$

$$n \times 1 \quad n \times m \quad m \times 1$$

- **Step 8 :**

  Let the output layer units, evaluate the output using sigmoidal function as

$$\{ O \}_O = \left\{ \begin{matrix} \\ \\ \dfrac{1}{( 1 + e^{-(I_{Oj})} )} \\ \\ \\ \end{matrix} \right\}$$

  Note : This output is the network output

# Back Propagation Network Algorithm

- **Step 9 :**

  Calculate the error using the difference between the network output and the desired output as for the $j^{th}$ training set as

$$E^P = \frac{\sqrt{\sum (Tj - Ooj)^2}}{n}$$

- **Step 10 :**

  Find a term { d } as

$$\{ d \} = \begin{Bmatrix} \\ \\ (Tk - Ook)\, Ook\, (1 - Ook) \\ \\ \\ \end{Bmatrix}_{n \times 1}$$

# Back Propagation Network Algorithm

- **Step 11 :**

  Find [ Y ] matrix as

$$\underset{m \times n}{[Y]} = \underset{m \times 1}{\{O\}_H} \; \underset{1 \times n}{\langle d \rangle}$$

- **Step 12 :**

  Find $$\underset{m \times n}{[\Delta W]^{t+1}} = \alpha \underset{m \times n}{[\Delta W]^{t}} + \eta \underset{m \times n}{[Y]}$$

- **Step 13 :**

  Find $$\underset{m \times 1}{\{e\}} = \underset{m \times n}{[W]} \; \underset{n \times 1}{\{d\}}$$

$$\{d^*\} = \begin{Bmatrix} - \\ - \\ e_i (O_{Hi})(1 - O_{Hi}) \\ - \\ - \end{Bmatrix}$$

$$\underset{m \times 1}{\quad} \quad \underset{m \times 1}{\quad}$$

  Find [ X ] matrix as

$$\underset{1 \times m}{[X]} = \underset{\ell \times 1}{\{O\}_I} \; \underset{1 \times m}{\langle d^* \rangle} = \underset{\ell \times 1}{\{I\}_I} \; \underset{1 \times m}{\langle d^* \rangle}$$

# Back Propagation Network Algorithm

- **Step 14 :**

  Find $$\underset{1 \times m}{[\Delta V]^{t+1}} = \alpha \underset{1 \times m}{[\Delta V]^{t}} + \eta \underset{1 \times m}{[X]}$$

- **Step 15 :**

  Find $$[V]^{t+1} = [V]^{t} + [\Delta V]^{t+1}$$

  $$[W]^{t+1} = [W]^{t} + [\Delta W]^{t+1}$$

- **Step 16 :**

  Find error rate as

  $$\text{error rate} = \frac{\Sigma E_p}{\text{nset}}$$

- **Step 17 :**

  Repeat steps 4 to 16 until the convergence in the error rate is less than the tolerance value

- **End of Algorithm**

  Note : The implementation of this algorithm, step-by-step 1 to 17, assuming one example for training BackProp Network is illustrated in the next section.

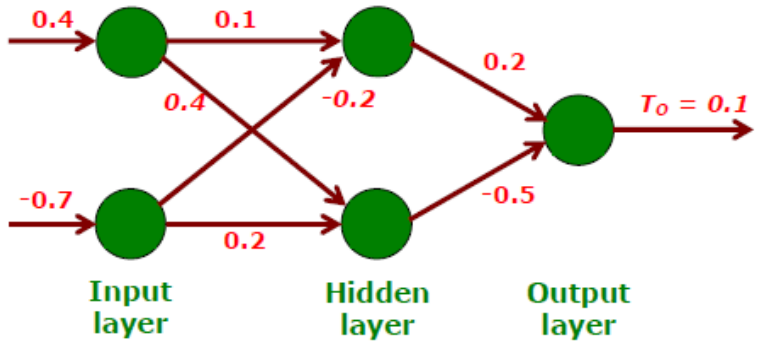
# Example Back Propagation Network

**Table : Training sets**

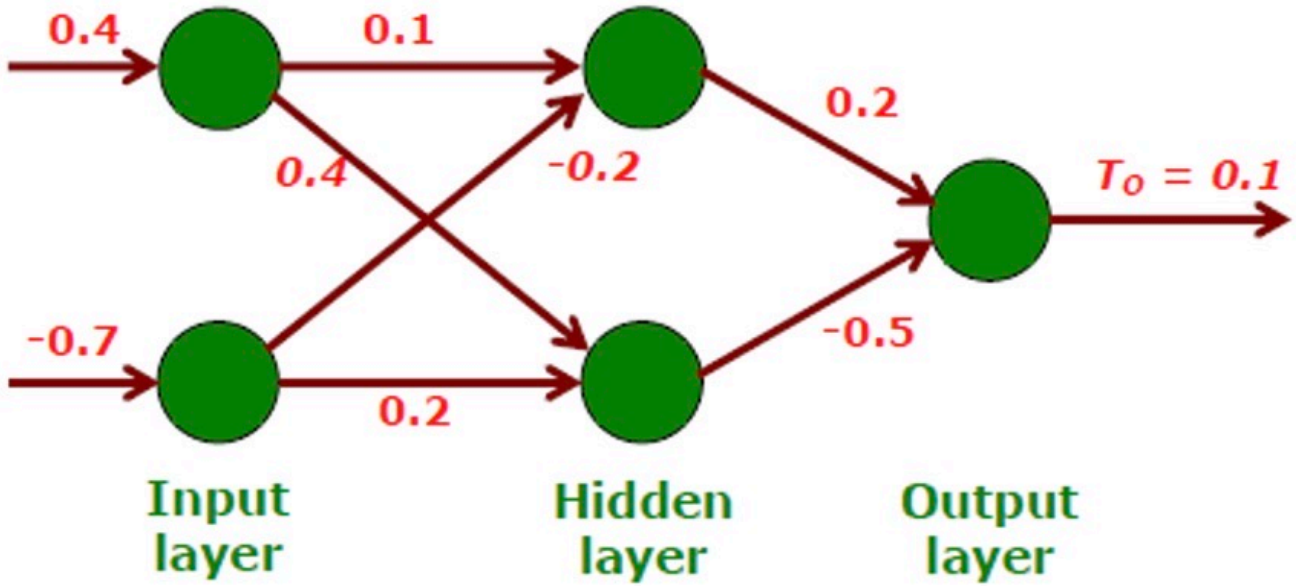
| S. No. | Input $I_1$ | Input $I_2$ | Output O |
|---|---|---|---|
| 1 | 0.4 | -0.7 | 0.1 |
| 2 | 0.3 | -0.5 | 0.05 |
| 3 | 0.6 | 0.1 | 0.3 |
| 4 | 0.2 | 0.4 | 0.25 |
| 5 | 0.1 | -0.2 | 0.12 |

In this problem,

- there are two inputs and one output.
- the values lie between **-1** and **+1** i.e., no need to normalize the values.
- assume two neurons in the hidden layers.
- the NN architecture is shown in the Fig. below.



# Example Back Propagation Network



# Example Back Propagation Network

Assume:

- Sigmoid activation: $\sigma(z) = \frac{1}{1+e^{-z}}$
- MSE loss: $L = \frac{1}{2}(t - \hat{y})^2$
- No bias terms, since none are shown
- Target $t = 0.1$
- Learning rate $= \eta$

Let $x_1 = 0.4$ and $x_2 = -0.7$.

# Example Back Propagation Network

## 1. Forward propagation

### Hidden neuron $h_1$

$$z_{h1} = (0.4)(0.1) + (-0.7)(-0.2)$$

$$z_{h1} = 0.04 + 0.14 = 0.18$$

$$h_1 = \sigma(0.18) = 0.5449$$

### Hidden neuron $h_2$

$$z_{h2} = (0.4)(0.4) + (-0.7)(0.2)$$

$$z_{h2} = 0.16 - 0.14 = 0.02$$

$$h_2 = \sigma(0.02) = 0.5050$$

# Example Back Propagation Network

**Output neuron**

$$z_o = (h_1)(0.2) + (h_2)(-0.5)$$

$$z_o = (0.5449)(0.2) + (0.5050)(-0.5)$$

$$z_o = 0.1090 - 0.2525 = -0.1435$$

$$\hat{y} = \sigma(-0.1435) = 0.4642$$

So, for this untrained network, the predicted output is:

$$\boxed{\hat{y} = 0.4642}$$

# Example Back Propagation Network

**Loss**

$$L = \frac{1}{2}(t - \hat{y})^2$$

$$L = \frac{1}{2}(0.1 - 0.4642)^2$$

$$\boxed{L = 0.0663}$$

# Example Back Propagation Network

## 2. Backpropagation: output layer

For sigmoid with MSE:

$$\delta_o = \frac{\partial L}{\partial z_o} = (\hat{y} - t)\hat{y}(1 - \hat{y})$$

$$\delta_o = (0.4642 - 0.1)(0.4642)(1 - 0.4642)$$

$$\boxed{\delta_o = 0.09058}$$

### Gradients of output weights

For weight $h_1 \to o$, initially $0.2$:

$$\frac{\partial L}{\partial w_{h1,o}} = \delta_o h_1$$

$$= (0.09058)(0.5449) = \boxed{0.04935}$$

For weight $h_2 \to o$, initially $-0.5$:

$$\frac{\partial L}{\partial w_{h2,o}} = \delta_o h_2$$

$$= (0.09058)(0.5050) = \boxed{0.04574}$$

# Example Back Propagation Network

## 3. Backpropagation: hidden layer

**Error term for hidden neuron $h_1$**

$$\delta_{h1} = \delta_o \cdot w_{h1,o} \cdot h_1(1 - h_1)$$

$$= (0.09058)(0.2)(0.5449)(1 - 0.5449)$$

$$\boxed{\delta_{h1} = 0.004492}$$

**Error term for hidden neuron $h_2$**

$$\delta_{h2} = \delta_o \cdot w_{h2,o} \cdot h_2(1 - h_2)$$

$$= (0.09058)(-0.5)(0.5050)(1 - 0.5050)$$

$$\boxed{\delta_{h2} = -0.011321}$$

# Example Back Propagation Network

## 4. Gradients of input-to-hidden weights

| Weight | Formula | Gradient |
|---|---|---|
| $w_{x1,h1} = 0.1$ | $\delta_{h1} x_1$ | $0.004492(0.4) = 0.001797$ |
| $w_{x2,h1} = -0.2$ | $\delta_{h1} x_2$ | $0.004492(-0.7) = -0.003145$ |
| $w_{x1,h2} = 0.4$ | $\delta_{h2} x_1$ | $-0.011321(0.4) = -0.004528$ |
| $w_{x2,h2} = 0.2$ | $\delta_{h2} x_2$ | $-0.011321(-0.7) = 0.007925$ |

# Example Back Propagation Network

## 5. One weight-update iteration

Gradient-descent rule:

$$w_{\text{new}} = w_{\text{old}} - \eta \frac{\partial L}{\partial w}$$

Thus:

$$w_{h1,o}^{new} = 0.2 - \eta(0.04935)$$

$$w_{h2,o}^{new} = -0.5 - \eta(0.04574)$$

$$w_{x1,h1}^{new} = 0.1 - \eta(0.001797)$$

$$w_{x2,h1}^{new} = -0.2 - \eta(-0.003145)$$

$$w_{x1,h2}^{new} = 0.4 - \eta(-0.004528)$$

$$w_{x2,h2}^{new} = 0.2 - \eta(0.007925)$$

# Example Back Propagation Network

For example, if $\eta = 0.1$, the updated weights after one iteration are:

| Weight | Old value | New value |
|---|---|---|
| $x_1 \to h_1$ | 0.100000 | 0.099820 |
| $x_2 \to h_1$ | -0.200000 | -0.199686 |
| $x_1 \to h_2$ | 0.400000 | 0.400453 |
| $x_2 \to h_2$ | 0.200000 | 0.199208 |
| $h_1 \to o$ | 0.200000 | 0.195065 |
| $h_2 \to o$ | -0.500000 | -0.504574 |

These new weights slightly reduce the loss for the target $0.1$.

# Loss Functions

- A loss function measures the difference between actual values and predicted values. Training aims to minimize this loss.

- **Mean Squared Error (MSE)**
- Used mainly for regression.

$$\text{MSE} = \frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$$

- Large errors receive a much larger penalty because errors are squared.
- Use cases: house-price prediction, temperature prediction, demand forecasting.

# Loss Functions

- **Mean Absolute Error (MAE)**
- Also, used for regression.

$$\text{MAE} = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

- MAE is less sensitive to outliers than MSE.
- Use cases: regression tasks where extreme errors should not dominate learning.

# Loss Functions

- **Cross-Entropy Loss**
- Used for classification problems.
- For Binary classification:

$$L_i = -\left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)\right]$$

- For Multiclass classification:

$$\text{BCE} = -\frac{1}{n} \sum_{i=1}^{n} \left[y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)\right]$$

# L2 Regularization

- L2 regularization is a technique used to reduce **overfitting** in a deep-learning model.
- It adds a penalty for large weight values to the normal loss function:

$$L_{\text{total}} = L_{\text{data}} + \lambda \sum_{l} \sum_{i} w_{l,i}^2$$

Where:

- $L_{\text{data}}$: original loss, such as Cross-Entropy or MSE
- $w_{l,i}$: a weight in layer $l$
- $\lambda$: regularization strength

# L2 Regularization

- The model therefore tries to achieve low prediction error while keeping weights reasonably small.
- Why it helps: very large weights can make a model overly sensitive to small changes in input data.
- L2 regularization discourages such extreme weights, producing a smoother model that generalizes better to unseen data.
- During gradient descent, the update becomes:

$$w \leftarrow w - \eta \left( \frac{\partial L_{\text{data}}}{\partial w} + 2\lambda w \right)$$

The term $2\lambda w$ continuously pulls each weight toward zero. This is why L2 regularization is often called **weight decay**.

- Example: Without L2: $w_{x1h1}$=0.0998, With L2: $w_{x1h1}$=0.09306

# Dropout

- Dropout randomly deactivates a fraction of neurons during training.
- For example, with dropout rate (0.5), approximately 50% of neurons are temporarily ignored in each training step.

- Benefits:
  - Prevents dependence on specific neurons
  - Reduces overfitting
  - Improves generalization
  - Dropout is used during training but not during inference/testing.

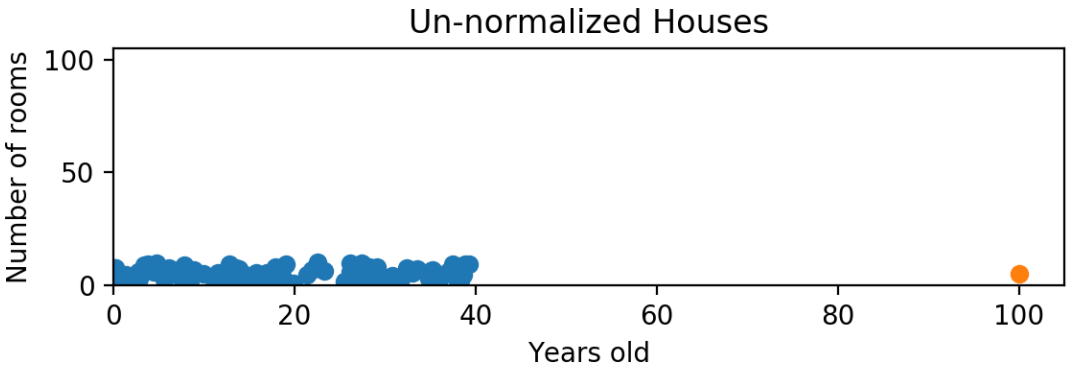
# Normalization: Standard Normalization

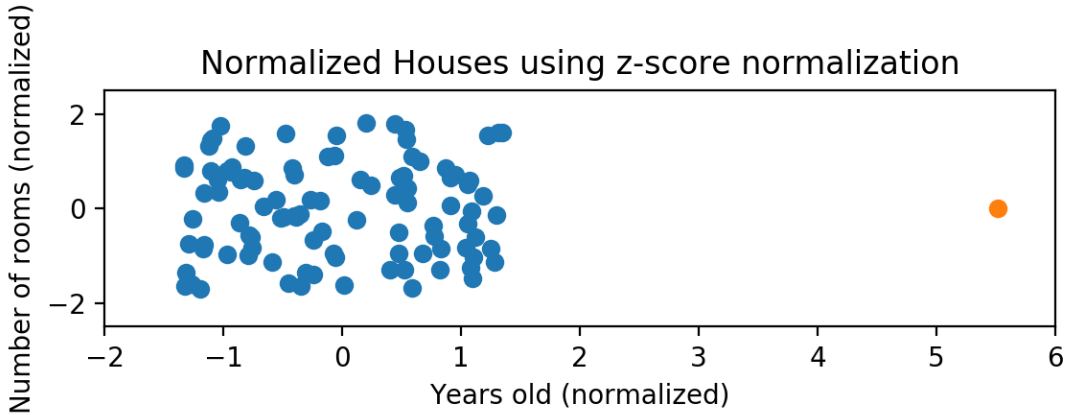
- Z-score normalization is a strategy of normalizing data that avoids this outlier issue. The formula for Z-score normalization is below:

$$Z_{Score}=(Value-\mu)/\sigma$$

- Here, μ is the mean value of the feature and σ is the standard deviation of the feature.
- If a value is exactly equal to the mean of all the values of the feature, it will be normalized to 0.
- If it is below the mean, it will be a negative number, and if it is above the mean it will be a positive number.
- The size of those negative and positive numbers is determined by the standard deviation of the original feature.

# Normalization: Standard Normalization



# ADAM Optimizer

- ADAM keeps two moving averages:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2$$

- Then it updates weights using:

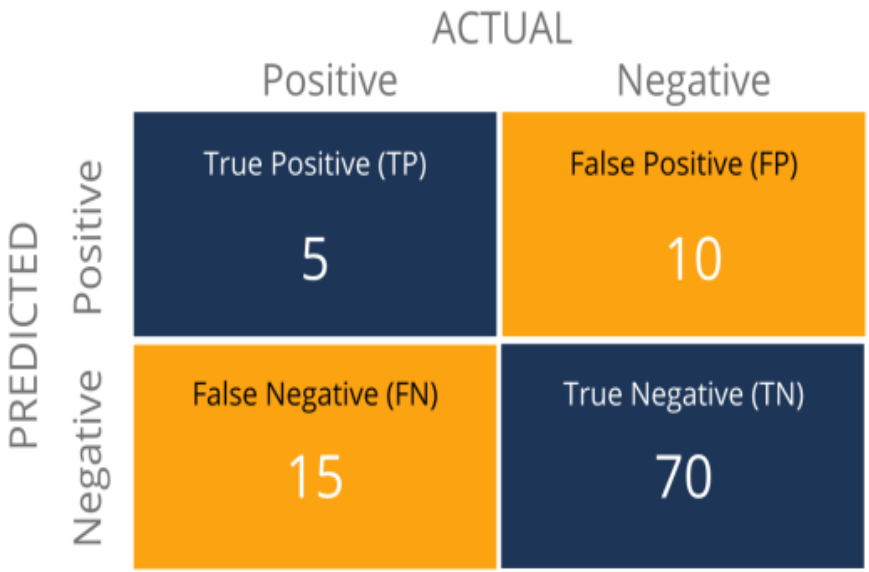
$$w_{t+1} = w_t - \eta \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon}$$

- $m_t$: momentum-like average of gradients
- $v_t$: average of squared gradients
- $\hat{m}_t, \hat{v}_t$: bias-corrected values

# SDG Vs ADAM Optimizer

| Aspect | SGD | Adam |
|---|---|---|
| Full form | Stochastic Gradient Descent | Adaptive Moment Estimation |
| Basic update | Uses the current gradient | Uses gradient plus adaptive history |
| Learning rate | Same learning rate for all parameters | Different effective learning rates for different parameters |
| Convergence | Usually slower | Usually faster initially |
| Memory needed | Low | Higher; stores moving averages |
| Hyperparameters | Mainly learning rate | Learning rate, beta_1, beta_2, epsilon |
| Suitable for | Simple models; large-scale vision tasks | Most deep-learning tasks; quick experimentation |
| Generalization | Can sometimes generalize better after careful tuning | Fast, but may sometimes generalize slightly worse |
| Sensitivity | Very sensitive to learning-rate choice | Less sensitive, but still needs tuning |

# Evaluation Metrics




$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN} = 0.75$$

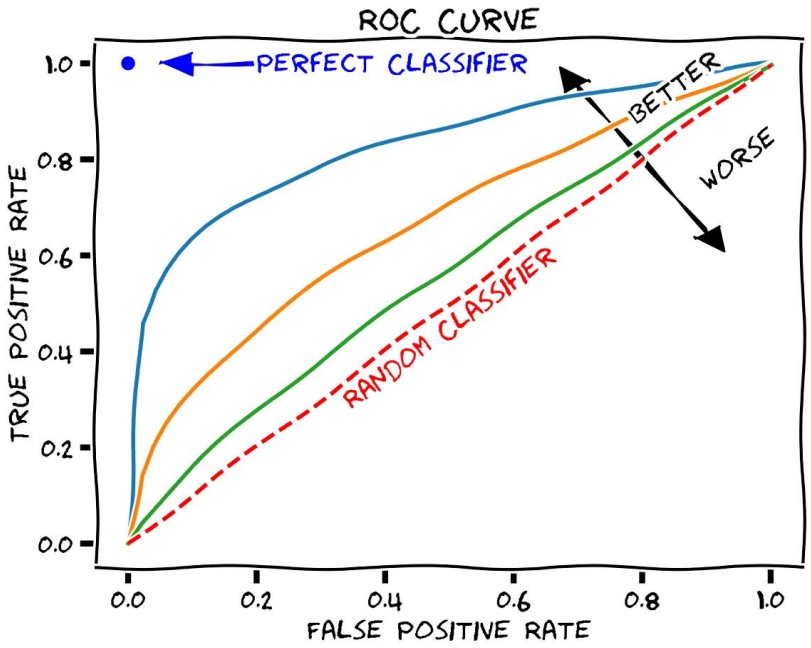
$$\text{Recall} = \frac{TP}{TP + FN} = 0.25$$

$$\text{Precision} = \frac{TP}{TP + FP} = 0.33$$

$$\text{F1 Score} = \frac{2 * \text{Precision} * \text{Recall}}{\text{Precision} + \text{Recall}} = 0.28$$

# AUC (Area Under the ROC Curve)

- An AUC-ROC curve is a graph that shows how well a binary classification model can tell two groups apart.
- It plots the **True Positive Rate** against the **False Positive Rate** at different probability threshold levels. The **AUC** score measures the total area under that graph, ranging from 0.5 for random guesses to 1.0 for a perfect model.



# AUC (Area Under the ROC Curve)

- For example, if: [ AUC=0.90 ], it means the model has about a 90% chance of assigning a higher fraud probability to a randomly selected fraudulent transaction than to a randomly selected genuine transaction.

- Unlike accuracy, AUC does not depend on one fixed threshold such as (0.5). This makes it especially useful for imbalanced fraud datasets.

| AUC value | Meaning |
|---|---|
| 1.0 | Perfect classifier |
| 0.9-1.0 | Excellent separation |
| 0.7-0.9 | Good separation |
| 0.5 | Random guessing |
| Below 0.5 | Worse than random |